

Como Funciona a Automação do Instagram

Explicação do fluxo configurado para publicar no Instagram com o computador desligado.

Projeto: `marcondesjm/cliente-x-instagram`

Conta alvo: `@marcondes.machado.official`

Atualizado em: 17/07/2026

Resumo rápido

A automação não depende mais do computador local. Ela roda no GitHub Actions, que é um ambiente na nuvem. O Supabase guarda os conteúdos dos carrosséis, o GitHub Actions busca esses conteúdos, gera as imagens, envia para o ImgBB e publica no Instagram usando a API oficial da Meta.

Estado atual: o fluxo foi testado em modo seguro, com `dry_run=true`, e passou com sucesso no GitHub Actions. Esse teste não publica no feed, mas comprova que a nuvem consegue executar a automação.

Quem faz o quê

Supabase

Guarda os posts que serão publicados. Foi criada a tabela `public.instagram_posts` com 5 carrosséis ativos para a conta `cliente-x`.

GitHub Actions

É o executor principal. Ele acorda nos horários programados, baixa o projeto, instala as dependências, gera os slides e chama as APIs externas. Como roda no GitHub, o computador pode ficar desligado.

ImgBB

Recebe as imagens geradas dos slides e devolve URLs públicas. A API do Instagram precisa dessas URLs para montar o carrossel.

Instagram Graph API

É a API oficial usada para criar os containers do carrossel e publicar o post final na conta `@marcondes.machado.official`.

Fluxo completo

1. Horário chega

O GitHub Actions dispara automaticamente conforme o cron configurado.

2. Slot é escolhido

Cada horário usa um slot diferente, para evitar repetir o mesmo carrossel no mesmo dia.

3. Conteúdo vem do Supabase

O script lê a tabela `instagram_posts`. Se o Supabase não estiver configurado, usa o JSON local como fallback.

4. Slides são renderizados

O Playwright/Chromium gera as imagens JPG dos slides.

5. Imagens sobem para ImgBB

O ImgBB devolve URLs públicas para cada slide.

6. Instagram monta o carrossel

A API cria os itens do carrossel e prepara o container principal.

7. Publicação final

No agendamento automático, o script chama `media_publish` e publica de verdade.

Horários configurados

Os horários estão em horário de Brasília:

- 11:50
- 13:00
- 16:00
- 19:00
- 22:00

No arquivo do GitHub Actions, esses horários aparecem em UTC:

- `50 14 * * *` - 11:50 Brasília
- `0 16 * * *` - 13:00 Brasília
- `0 19 * * *` - 16:00 Brasília
- `0 22 * * *` - 19:00 Brasília
- `0 1 * * *` - 22:00 Brasília

Secrets usados

As chaves sensíveis não ficam no código. Elas ficam em **GitHub Actions Secrets**:

- `CLIENTE_X_INSTAGRAM_ACCESS_TOKEN`
- `CLIENTE_X_INSTAGRAM_USER_ID`
- `IMGBB_API_KEY`
- `SUPABASE_URL`
- `SUPABASE_SERVICE_ROLE_KEY`

Importante: esses valores não devem ser colocados em arquivos públicos nem enviados no chat. O arquivo

`.env` local serve só para testes no seu computador.

Diferença entre teste e publicação real

Teste manual

Quando o workflow é rodado manualmente com `dry_run=true`, ele cria imagens, sobe para ImgBB e prepara o carrossel no Instagram, mas não chama a publicação final. Isso é o teste seguro.

Agendamento automático

Quando o horário automático dispara, o workflow usa `DRY_RUN=false`. Nesse caso, depois de preparar o carrossel, ele publica no feed.

Arquivos principais

- `.github/workflows/instagram-feed-cliente-x.yml` - agenda e execução na nuvem.
- `automation/instagram-template/scripts/publish-carousel.mjs` - lógica de gerar, subir e publicar.
- `supabase/instagram-posts.sql` - criação da tabela e posts iniciais no Supabase.
- `automation/instagram-template/config/content-packs.json` - fallback local dos posts.

Como saber se funcionou

1. Acesse o repositório no GitHub.
2. Entre em **Actions**.
3. Abra o workflow **Instagram Feed Cliente X**.
4. Confira se o run do horário ficou com status **success**.
5. Se quiser detalhes, abra o artifact `instagram-template-runs`.

Conclusão: com GitHub Actions e Supabase configurados, a automação pode publicar mesmo com o computador desligado.